

OMNIVERSAL ARCHITECT LABS

Adversarial Cross-Review and Iterative Improvement Protocol

A Reusable Framework for Independent Critique, Bounded Testing, Cross-Model Review, and Versioned Improvement

Document ID	OAL-ACR-CHATGPT-001
Version	v0.1
Date	October 1, 2026
Prepared by	ChatGPT (OpenAI)
Prepared for	Nathan Lewis II, Architect of Record
Organization	Omniversal Architect Labs (OAL)
Status	PROPOSED — NOT CANON — NOT ONESTAMP-RATIFIED
Publication intent	GitHub publication and multi-AI comparative review

Prepared as an independent ChatGPT contribution for team pass-through, comparison, criticism, and future iteration.

Document Control

Field	Value
Document owner	Nathan Lewis II, Architect of Record
Preparer	ChatGPT (OpenAI)
Initial release	October 1, 2026 — v0.1
Review status	Proposed working protocol; open to criticism and comparative revision
Canon status	Not canon; not ONESTAMP-ratified
Primary use	Repeatable adversarial review of OAL frameworks, documents, prompts, systems, repositories, simulations, policies, and collaborative AI outputs
Source basis	The End Cycle Laboratory adversarial cross-review materials supplied by the Architect, the accompanying pilot simulation artifacts, and the Architect's stated goals for a reusable process
Intended outputs	Versioned DOCX/PDF review documents, machine-readable registers where useful, reproducibility artifacts, and GitHub-ready iteration records

Version History

Version	Date	Preparer	Change summary	Status
v0.1	October 1, 2026	ChatGPT (OpenAI)	Initial independent protocol: mission, review architecture, tiering, registers, test workflow, cross-model lanes, quality gates, and commission prompt.	PROPOSED

Status Notice

STATUS: This document is a proposal for review. It does not establish OAL canon, policy, authority, or ratification. Adoption, modification, or rejection remains subject to Architect review and any applicable ONESTAMP process.

Independence and Disclosure

This document is an independent ChatGPT rendering of a reusable process inspired by the supplied End Cycle Laboratory cross-review and the Architect's request. It is not presented as a neutral synthesis of all future model outputs. It should be compared against independent versions from Perplexity, Claude, Gemini, Microsoft Copilot, Grok, human reviewers, and any later collaborators.

The protocol intentionally allows disagreement. A later reviewer may reject its tier model, labels, governance assumptions, test requirements, or document structure. Such disagreement should be recorded in the discrepancy register rather than removed for cosmetic consistency.

Table of Contents

Right-click and select “Update Field” if the table of contents does not appear.

Executive Summary

OAL has generated a substantial body of conceptual, technical, narrative, governance, and collaborative AI documentation. The central risk is no longer merely a shortage of ideas. It is the possibility that documentation volume, agreement among collaborators, or elaborate framework design may be mistaken for evidence that a system has been exercised, evaluated, or shown to work.

This protocol converts adversarial review into a repeatable, version-controlled practice. It requires reviewers to declare the corpus they actually read; distinguish source claims, executed findings, external evidence, assessments, proposals, open questions, limitations, decisions, hygiene issues, and work not run; preserve disagreements; identify assumptions and missing tasks; propose or execute the smallest test appropriate to the claim; and convert results into explicit revision actions.

The protocol does not seek consensus. Independent convergence may be informative, but agreement is not proof. Disagreement is treated as structured work: some conflicts should be tested, some require an Architect decision, some arise from version or naming problems, and some should remain open, deferred, quarantined, or retired.

A tiered review model prevents the process from becoming another source of documentation bloat. Lightweight artifacts receive lightweight reviews; mature or high-impact systems require preregistration, controls, reproducible results, dissent preservation, and a clear post-review change record. Not every topic requires code or simulation, but every major claim should be matched to the smallest legitimate validation method available.

For multi-AI collaboration, the recommended approach combines a common core with differentiated lanes. Each system receives the same source packet, metadata rules, epistemic labels, and non-canon notice, while carrying a distinct burden—such as evidence verification, measurement design, ambiguity preservation, technical feasibility, implementation workflow, or red-team analysis. Independent first passes precede cross-review so that the team can distinguish genuine convergence from prompt-induced similarity.

Central principle: A framework is not mature merely because it is detailed. It becomes more mature when it can state what would count as failure, run a bounded test, report what happened, and change accordingly.

Mission and Directive

Mission Statement

MISSION: The Adversarial Cross-Review and Iterative Improvement Protocol exists to make OAL-generated work inspectable, testable, comparable, and revisable before it is treated as stable, operational, or canonical. It does not seek consensus as proof of correctness. It identifies assumptions, disagreements, omissions, unperformed tasks, and falsifiable claims; records them without averaging them away; and converts them into bounded tests, decisions, hygiene corrections, or explicit open questions.

Operating Directive

DIRECTIVE: Review the work as if it may contain important insights and important failures at the same time. Preserve useful structures, identify unsupported claims, test the smallest meaningful part, report failure honestly, and do not allow documentation volume, AI agreement, institutional formality, or authorial investment to stand in for evidence.

Objectives

- Create a repeatable method for reviewing one document, a family of versions, or a repository-scale body of work.
- Separate what the corpus says from what a reviewer concludes and from what an executed exercise actually finds.
- Expose assumptions, dependencies, internal contradictions, omissions, edge cases, and unperformed tasks.
- Turn disagreement into traceable tests, decisions, hygiene corrections, or explicitly unresolved questions.
- Require evidence proportionate to the claim while avoiding unnecessary simulations or procedural overhead.
- Preserve criticism, dissent, null results, and failed tests as useful iteration records.
- Support GitHub publication through stable identifiers, version history, cross-links, and reproducible artifacts.
- Enable multiple AI systems and human contributors to produce genuinely independent, comparable reviews.

Non-Goals

- The protocol does not determine whether a project is “good” or “bad” as a single global judgment.
- It does not make agreement among AI systems, reviewers, or source documents evidence of correctness.
- It does not make a proposal canon, operational policy, or institutional authority.
- It does not require Python, simulation, or quantitative testing where another validation method is more appropriate.
- It does not retroactively rewrite source artifacts or conceal conflicting versions.
- It does not use adversarial review as permission for personal attacks, motive attribution, or dismissive criticism.
- It does not treat an internal model, tabletop exercise, or fictional scenario as direct proof about the real world.
- It does not require every open question to be resolved in the current iteration.

Protocol Overview

The protocol has seven linked functions: corpus control, epistemic separation, independent critique, discrepancy management, test design, iterative action, and publication traceability. These functions may be scaled up or down according to the review tier, but they should remain conceptually distinct.

Function	Core question	Required output
Corpus control	What was actually reviewed, and what was missing?	Source register and declared-input list
Epistemic separation	What is a source claim, finding, evidence item, assessment, proposal, or unresolved issue?	Consistent labels and claim discipline
Independent critique	What does each artifact contribute, assume, omit, contradict, or leave undone?	Artifact-by-artifact adversarial review
Discrepancy management	Which conflicts should be tested, decided, cleaned up, deferred, quarantined, retired, or preserved?	Discrepancy register
Test design	What is the smallest legitimate exercise capable of challenging the claim?	Preregistration and validation plan
Iterative action	What specifically changes because of the review?	Improvement register and decision log
Publication traceability	Can a later reader reproduce the record and understand version relationships?	Stable IDs, change log, linked artifacts, and status notices

Lifecycle

1. Commission the review with a stable ID, declared scope, source packet, review tier, and non-canon status.
2. Freeze or identify the corpus window so reviewers know what version or commit range they are assessing.
3. Conduct independent first-pass reviews before sharing newly generated reviews among collaborators.
4. Create source, assumption, dependency, discrepancy, testability, and action registers.
5. Select the smallest high-leverage tests or validation tasks; preregister them before execution.
6. Execute, preserve raw artifacts, and label results PASS, FAIL, MIXED, INDETERMINATE, or NOT RUN.
7. Cross-review the reviews and results, documenting revised or withdrawn positions.
8. Make evidence-resolvable changes, elevate value or authority questions for decision, and preserve unresolved dissent.
9. Publish the record with a “What changed because of this review?” section and commission the next iteration only when warranted.

Definition of Adversarial

“Adversarial” means actively seeking conditions under which a claim, mechanism, interpretation, score, workflow, or governance rule would fail. It requires fair representation of the source, specific criticism, explicit evidence standards, and willingness to revise the reviewer’s own position. It does not mean hostility toward authors or reflexive rejection of the work.

Review Tier Model

The review tier should be selected before work begins. The tier controls expected depth, validation burden, artifact requirements, and stopping rules. A document may advance to a higher tier when its scope, maturity, risk, authority, or intended use increases.

Tier	Best use	Typical output	Minimum validation expectation
Tier 0 — Intake and catalog	Fragments, prompts, notes, small early documents	One- to two-page review card	No exercise required; identify claim type, dependencies, and next disposition
Tier 1 — Adversarial document review	Single framework, policy, architecture, or concept	Focused review dossier	At least one proposed falsifier, verification task, or edge-case test
Tier 2 — Cross-review	Multiple AI versions or competing artifacts on one subject	Cross-version review with registers	At least one bounded pilot, tabletop, source audit, dataset check, or controlled comparison where feasible
Tier 3 — Execution dossier	High-value recurring frameworks or public-facing systems	Evidence dossier plus reproducibility artifacts	Preregistered tests, controls, results, limitations, and revision decisions
Tier 4 — Canon or operational decision	ONESTAMP consideration, institutional use, material deployment	Ratification or operational decision packet	Complete evidence trail, dissent, known limits, rollback conditions, and post-adoption review trigger

Tier Selection Criteria

- Impact: What happens if the framework is wrong, incomplete, or misapplied?
- Authority: Will the artifact guide canon, policy, resource allocation, access, or operational behavior?
- Maturity: Is the work exploratory, specified, prototyped, executed, or already in use?
- Complexity: How many artifacts, systems, actors, assumptions, and dependencies are involved?
- Reversibility: Can errors be corrected cheaply, or will implementation create durable consequences?
- Evidence burden: Does the claim require source verification, user observation, code execution, simulation, or real-world evaluation?
- Publication exposure: Is the artifact private, internally published, broadly public, or intended for external reliance?

Anti-Bloat Rule

RULE: The review process must remain proportionate to the decision it informs. If a review cycle does not produce a clarified claim, a new test, an executed result, a repaired artifact, a decision, or a justified preserve/quarantine/retire action, it should not automatically expand into another document cycle.

Corpus and Provenance Control

No review is stronger than its declared inputs. Every reviewer must identify what was read in full, partially reviewed, unavailable, referenced secondhand, or excluded. Missing artifacts are findings about provenance and reproducibility; they are not evidence of the missing artifact's contents.

Required Source Register

Field	Required entry
Source ID	Stable short identifier such as S-01
Artifact title	Exact title shown in the source
Preparer or author	Named person, AI system, team, or unknown
Date and version	Exact metadata; note conflicts between file and document metadata
Artifact type	Prompt, memo, framework, code, dataset, simulation, policy, report, transcript, or other
Claimed stage	What the artifact says it delivers
Observed stage	What is actually present
Repository location	URL, path, branch, tag, or commit when available
Review completeness	Full, partial, metadata only, inaccessible, or secondhand
Dependencies and gaps	Missing attachments, code, data, citations, prior versions, or ambiguous relationships

Corpus Freeze

For cross-model review, define a corpus window using a dated source packet, release tag, branch, or commit range. New artifacts may be added later, but they should not silently alter the basis of an already issued review. A supplemental review can document the effect of new material.

Provenance Exceptions

- If a file is missing, label it MISSING and explain how that limits the review.
- If only a summary is available, label claims from it SECONDHAND and do not attribute details beyond the summary.
- If authorship is unclear, record UNKNOWN rather than inferring a preparer.
- If two files claim the same ID or version, open a HYGIENE discrepancy.
- If code cannot be executed, distinguish code inspection from execution findings.
- If a source changes during review, preserve or identify the reviewed version and open a supplemental source record.

Epistemic Labeling Standard

The following labels prevent source language, reviewer judgment, executed output, and future proposals from collapsing into one authoritative voice. Each substantive claim in a review should be written so its epistemic status is clear from wording, section placement, or an explicit label.

Label	Meaning	Example use
SOURCE	A claim, instruction, design element, or quotation contained in a named source artifact.	SOURCE: Version A assigns approval authority to the Architect.
FINDING	A result produced by an executed exercise, analysis, simulation, prototype, or test; valid only under the stated method and assumptions.	FINDING: Under preregistered parameters, the control produced more false approvals.
EVIDENCE	A claim supported by an external cited source, dataset, official record, or verifiable artifact.	EVIDENCE: Repository history shows the identifier first appeared in commit X.
ASSESSMENT	The reviewer’s reasoned evaluation or adversarial judgment.	ASSESSMENT: The shutdown rule is underspecified.
PROPOSAL	A new option, design, process, test, or revision recommended for a later iteration.	PROPOSAL: Add independent scoring and an appeal path.
OPEN	A question not resolvable using available materials.	OPEN: No source defines which version controls when IDs conflict.
LIMITATION	A constraint on interpreting a claim, method, or result.	LIMITATION: The pilot used one author and synthetic scenarios.
DECISION	A value, canon, authority, scope, or priority question reserved for the Architect or designated authority.	DECISION: Choose whether the role is advisory or binding.
HYGIENE	A documentation, naming, attribution, versioning, provenance, citation, or repository correction.	HYGIENE: Two files carry the same document ID.
NOT RUN	A planned exercise or task that was not executed, with the reason recorded.	NOT RUN: No independent evaluator was available in the review window.

Claim Language

- Use “supports,” “fails to support,” “is consistent with,” “was observed under these assumptions,” or “remains unresolved.”
- Reserve “proves” for formal proof under explicit assumptions.
- Do not call a predicted result, designed behavior, or expected output a finding.
- Do not turn an analogy into evidence without explaining the mapping and its limits.
- Do not treat the absence of contrary evidence as positive validation.
- State when a conclusion depends materially on a parameter, prompt, evaluator, dataset, or author judgment.

Independent Review Method

Each artifact should receive a fair summary before critique. The reviewer should identify unique contributions and strengths as carefully as weaknesses. A criticism is strongest when it names the affected claim or mechanism, explains the consequence, and proposes a way to test, decide, clean up, defer, quarantine, preserve, or retire it.

Required Questions

1. What does this source contribute that the other sources do not?
2. What does it make clearer, safer, more executable, or more useful?
3. What does it assume rather than test?
4. Which claims are unsupported, ambiguous, internally inconsistent, or overconfident?
5. Which definitions, thresholds, roles, procedures, records, or exit conditions are missing?
6. What could fail if the framework were applied as written?
7. Which dependencies, stakeholders, edge cases, incentives, or abuse paths are omitted?
8. What tasks does the source claim or imply should be done but has not done?
9. Can the central claims be tested now? If yes, how? If not, why not?
10. What should the next iteration preserve, revise, quarantine, defer, or retire?

Critique Standard

Weak critique	Stronger adversarial formulation
"This is confusing."	"The document uses 'approval' for both advisory review and binding authorization; this creates an authority conflict in Sections X and Y. Open a terminology and decision discrepancy."
"There is no evidence."	"The source predicts a reduction in error but reports no baseline, executed comparison, or acceptance threshold. Classify the statement as PROPOSAL until X-01 is run."
"The model is biased."	"The scenario generator and evaluator are the same system, creating circularity in classification claims. Require blind or independent scoring."
"The framework is too large."	"The required workflow has 14 stages but no stopping rule; overhead may exceed the value of review for Tier 0-1 artifacts. Test a reduced workflow against completion time and defect detection."
"I disagree."	"The disagreement concerns who holds final authority, which is not empirically resolvable. Record it as DECISION AD-01 rather than forcing consensus."

Self-Critique Requirement

A reviewer must apply the same standard to its own prior output. If new evidence, another reviewer, or an executed test undermines an earlier conclusion, the reviewer should explicitly retain, revise, or withdraw the position and state why.

Assumption and Dependency Audit

The audit surfaces conditions that must hold for the framework to function as intended. It should include explicit assumptions and assumptions embedded in definitions, workflow order, data access, authority, incentives, tooling, or expected participant behavior.

ID	Assumption or dependency	Where it appears	Type	Why it matters	Support status	Failure mode	Resolution
A-01	[Exact assumption]	[Source/section]	Technical / governance / ethical / narrative / economic / operational	[Consequence]	SOURCE / EVIDENCE / OPEN	[What fails if wrong]	TEST / DECISION / HYGIENE / DEFER

Required Audit Categories

- Definitions, identity criteria, and category boundaries.
- Scope limits and non-goals.
- Authority, delegation, escalation, appeal, and decision rights.
- Data quality, availability, lineage, retention, and access.
- Resources, staffing, budget, time, compute, and maintenance.
- Incentives, gaming, institutional capture, and conflicts of interest.
- Stakeholder representation, consent, accessibility, and affected nonparticipants.
- Edge cases, adversarial use, failure states, and recovery paths.
- Reversibility, rollback, shutdown, archival, and record loss.
- Safety, privacy, legal, ethical, and canon constraints where relevant.
- Dependencies on models, platforms, APIs, vendors, repositories, or external systems.
- Assumptions inherited from earlier prompts, documents, or unverified summaries.

Discrepancy Management

Disagreements should not be averaged into vague consensus. Every material discrepancy should preserve the competing positions, explain why the difference matters, identify what is known, and receive a resolution path.

ID	Discrepancy	Positions	Why it matters	Evidence/test status	Resolution path	Owner
D-01	[Specific conflict]	[Version A / Version B / reviewer]	[Consequence]	[Known, unknown, or test status]	TEST / DECISION / HYGIENE / DEFER / QUARANTINE / RETIRE / PRESERVE	[Architect / collaborator]

Resolution Paths

Path	Meaning
TEST	The conflict can be narrowed through a predefined exercise, prototype, source analysis, or comparison.
DECISION	The conflict concerns values, canon, authority, scope, priorities, or design choice and requires designated human authority.
HYGIENE	The conflict arises from naming, metadata, attribution, versioning, registry, citation, or repository problems.
DEFER	The issue matters but cannot responsibly be resolved in the current iteration.
QUARANTINE	The concept may be useful but should not shape shared assumptions until tested or independently reviewed.
RETIRE	The claim, mechanism, or rule failed a stated test or is no longer justified.
PRESERVE	The component is retained provisionally, with its limits and review trigger documented.

No Silent Resolution

A discrepancy closes only when the record links to the test result, Architect decision, hygiene correction, retirement rationale, or other closure artifact. Rewording two positions until they appear compatible does not close the discrepancy unless the original substantive conflict is actually resolved.

Testability and Validation

Validation should be matched to the type of claim. A software claim may require execution and unit tests; a provenance claim may require repository history; a classification claim may require blinded raters; a governance claim may require tabletop scenarios and incentive analysis; a narrative or canon question may require an Architect decision rather than empirical testing.

Testability Status

Status	Definition
Testable now	A feasible bounded method and required materials are available.
Partly testable	Some components can be tested, but central real-world, normative, metaphysical, or long-horizon claims cannot.
Not testable now	Data, access, conditions, authority, or valid methods are unavailable in the current iteration.
Not empirically testable	The claim is definitional, philosophical, aesthetic, canonical, or normative; classify it accordingly instead of presenting it as empirical.

Validation Methods

Method	Best suited to	Primary caution
Tabletop scenario	Governance, policy, workflows, handoffs, crises	Facilitator assumptions may determine the outcome.
Source audit	Evidence, citations, provenance, historical claims	Absence from searched sources is not proof of nonexistence.
Red-team prompt test	AI behavior, robustness, safety, instruction hierarchy	Prompt sample and model version may limit generalization.
Reproducibility check	Code, simulations, analyses, build pipelines	Reproducing output does not validate underlying assumptions.
Version-difference analysis	Competing documents and evolving specifications	Textual difference does not establish practical significance.
Dependency-map review	Architectures, organizations, repositories	Hidden dependencies may remain undiscovered.
User or stakeholder journey	Usability, access, burden, handoffs	Small or unrepresentative samples can mislead.
Blinded classification	Scoring, detection, distinguishability	Requires credible ground truth and rater agreement analysis.
Data-quality check	Datasets, registries, metrics, reporting	A clean dataset can still measure the wrong construct.
Counterfactual comparison	Interventions, rules, alternative designs	Counterfactual assumptions must be explicit.
Code test suite	Software behavior and regressions	Passing tests only covers encoded cases.
Policy stress test	Edge cases, incentives, abuse, emergency behavior	Scenario selection can bias conclusions.
Timeline/provenance audit	Authorship, attribution, sequence, identifiers	Incomplete history limits certainty.
Internal-consistency test	Definitions, authority, stages, cross-references	Consistency does not establish real-world effectiveness.
Prototype acceptance test	Implementation feasibility and specified behavior	Prototype context may not reflect operational scale.

Minimum rule: Run the smallest test appropriate to the claim. Do not require code when a source audit, tabletop, blinded review, or Architect decision is the legitimate method.

Preregistration and Exercise Design

Tier 2 or higher reviews should execute at least one appropriate bounded exercise unless the review documents why no legitimate exercise can be run. Preregistration occurs before execution so that success criteria, parameters, and interpretations are not rewritten after results are known.

Preregistration Block

Field	Required content
Test ID	Stable identifier such as X-01
Purpose	The decision or uncertainty the test informs
Claim or hypothesis	Exact proposition being challenged
Falsifier	Outcome that would materially challenge the claim
Inputs	Documents, data, prompts, code, scenarios, or participants
Parameters	Assumptions, thresholds, seeds, model versions, and configuration
Baseline/control	No-intervention condition, current practice, alternate rule, or benchmark
Comparison condition	Competing design, version, or counterfactual
Metrics	Measures and calculation rules
Decision thresholds	PASS, FAIL, MIXED, or INDETERMINATE criteria
Roles	Generator, evaluator, red-team reviewer, and decision owner
Limitations	Known threats to validity and prohibited generalizations
Date/version	Preregistration date and immutable version or commit
Artifacts	Expected output files and repository locations

Quality Rules

- Use a baseline, control, counterfactual, or comparison branch whenever feasible.
- Separate scenario generation from evaluation for detectability, classification, ranking, or indistinguishability claims.
- Do not let the author of the expected answer be the only scorer.
- Measure both benefit and harm when evaluating governance or trade-off claims.
- Disclose whether results depend materially on chosen parameters or prompt wording.
- Preserve raw outputs, code, tables, prompts, configurations, and model/version metadata where applicable.
- Report errors, null results, failed runs, protocol deviations, and indeterminate outcomes.
- Do not change thresholds after execution without creating an explicitly labeled exploratory analysis.

Solo-Creator Fallbacks

When a fully independent evaluator is unavailable, partial safeguards include sealed ground-truth files, delayed reveal, randomized case ordering, a second model that did not generate the cases, blinded scoring, predefined rubrics, or a later human audit. These do not create full independence; the limitation must remain visible.

Results and Interpretation Standard

Status	Definition	Required interpretation
PASS	The result matched the preregistered prediction under stated conditions.	State the narrow support and do not generalize beyond method and assumptions.
FAIL	The result contradicted the preregistered prediction under stated conditions.	Identify which claim or mechanism is challenged and what should change.
MIXED	Measures, cases, branches, or parameter ranges produced conflicting results.	Separate the conditions and avoid collapsing them into an average conclusion.
INDETERMINATE	The exercise ran but could not discriminate among relevant alternatives.	Explain whether the method, data, construct, power, or ground truth was inadequate.
NOT RUN	The planned exercise was not executed.	State why, what was completed, and whether the test remains valid for a later iteration.

Finding Boundaries

- A PASS does not prove the theory or framework.
- A FAIL does not automatically invalidate the entire project.
- A simulation result describes the implemented model, not reality by default.
- A reproducible result can still rest on invalid assumptions.
- A null or indeterminate result can reveal that a metric, scenario, or distinction is inadequate.
- Post-hoc exploration may generate new hypotheses but must not be relabeled as preregistered confirmation.

Iterative Improvement and Decisions

The review is incomplete until analysis becomes traceable work. Every material weakness, discrepancy, failed test, or open dependency should either produce an action, a decision request, an explicit defer/quarantine record, or a documented reason no action is warranted.

Improvement Register

ID	Action	Trigger	Priority	Owner	Closure evidence	Target iteration	Status
I-01	[Specific action verb + object]	D-03 / A-04 / X-02	High / Medium / Low	[Assigned owner]	[Required artifact or result]	I{n}	OPEN

Action Language

Use concrete verbs: define, separate, test, compare, rename, register, preregister, audit, prototype, red-team, archive, retire, quarantine, document, translate, pause, or escalate. Avoid actions such as “improve the framework,” “consider the issue,” or “make it better.”

Architect Decision Log

ID	Decision required	Options	Consequences	Recommendation	Default if undecided	Trigger/deadline
AD-01	[Value, canon, scope, authority, or priority question]	A / B / C	[Trade-offs]	[Optional]	[Provisional state]	[When needed]

Only questions that cannot be resolved through testing, source verification, or documentation hygiene should be elevated as Architect decisions. The protocol should reduce decision burden rather than transfer every uncertainty upward.

What Changed Record

Every completed cycle should include a concise record of what changed because of the review: definitions revised, claims downgraded, tests added, mechanisms changed, files renamed, decisions made, items quarantined or retired, and questions still open. If nothing changed, the cycle should explain what value it produced and why another cycle is justified.

Multi-AI Collaboration Workflow

Independent model versions are most useful when they share enough structure to compare but retain different analytical burdens. Giving every model the same highly detailed reasoning prompt may create artificial convergence. The recommended design uses a common source packet and output core, paired with differentiated review lanes.

Common Source Packet

- Exact topic, research question, and review tier.
- Frozen source corpus or repository commit range.
- Source register and known missing artifacts.
- Stable document ID and versioning rules.
- Epistemic labels and result-status definitions.
- Required registers and quality gates.
- Status notice that outputs are proposed, not canon, and not ONESTAMP-ratified.
- Rule that every input read must be disclosed.
- Rule that agreement is not evidence and criticism must remain specific and constructive.

Suggested Independent Lanes

Lane	Primary burden
Perplexity	Evidence audit, source verification, real-world analogies, testability, and translation implications.
ChatGPT	Measurement design, controls, scoring instruments, falsifiers, synthesis, and cross-version consistency.
Claude	Non-assumptions, ambiguity preservation, ethics, scope boundaries, and restraint against over-institutionalization.
Gemini	Systems architecture, technical feasibility, implementation dependencies, data flows, and integration risks.
Microsoft Copilot	Operationalization, documentation hygiene, implementation planning, version control, and developer workflow.
Grok	Red-team critique, incentive failures, institutional capture, abuse cases, edge cases, and adversarial scenarios.
Human Architect	Canon, priorities, aesthetic intent, authority boundaries, values, and final ratification status.

These lanes are starting assignments, not permanent claims about model capability. Rotate or modify lanes across iterations to detect role-induced blind spots and model drift.

Six-Phase Workflow

1. Commission: issue the shared packet, lane assignment, review tier, output requirements, and immutable source window.
2. Independent first pass: each collaborator produces its review before reading the other newly commissioned reviews.
3. Cross-review: collaborators identify independent convergence, prompt-induced convergence, disagreement, revised positions, and withdrawn recommendations.
4. Prioritization: select one to three high-leverage tests or validation tasks rather than attempting every proposed exercise.
5. Execution: preregister, run, preserve artifacts, and report PASS, FAIL, MIXED, INDETERMINATE, or NOT RUN.
6. Decision and archive: make required decisions, update registers, publish the “What changed” record, preserve dissent, and commission another iteration only when justified.

Cross-Review Questions

- Where do reviewers agree for independent reasons?
- Where do they agree because they share a source, prompt, rubric, or unexamined assumption?
- Where do they disagree, and is the disagreement empirical, definitional, normative, canonical, or procedural?
- Which disagreement can be tested?

- Which disagreement requires an Architect decision?
- Which disagreement is a naming, attribution, metadata, or versioning problem?
- Which prior positions should each reviewer retain, revise, or withdraw after seeing the other analyses?

Stage Gates and Quality Gates

Iteration Gates

Gate	Purpose	Required output	Stop rule
G1 — Revise	Identify what to preserve, revise, reject, quarantine, or defer.	Revision memo, corpus register, and discrepancy positions.	No testing until the claims and review scope are explicit.
G2 — Preregister	Define hypotheses, falsifiers, inputs, parameters, controls, metrics, and roles.	Versioned preregistration artifact.	No confirmatory execution without a dated preregistration record.
G3 — Execute	Run the smallest tests capable of challenging central claims.	Raw artifacts, code/data where relevant, results, failures, and limitations.	No institutional or canon recommendation based solely on planned work.
G4 — Translate	Convert evidence into design, policy, operational, narrative, or canon options.	Implementation options, trade-offs, and dissent record.	Do not present analogy or internal simulation as direct real-world proof.
G5 — Review	Adversarially review the new results and revisions.	Contradiction, limitation, and revision register.	No ratification without unresolved-risk disclosure and rollback/review triggers.

Submission Failure Conditions

- Inputs read are not declared.
- A proposed or predicted result is described as an executed finding.
- No source window, version, or document identifier is provided.
- Material disagreements are erased or averaged without a recorded resolution.
- A Tier 2+ dossier has neither an executed exercise nor a documented reason no legitimate exercise could be run.
- A classification or scoring claim relies solely on the same unblinded creator and evaluator without disclosure.
- Limitations are omitted or placed so far from findings that readers are likely to overgeneralize.
- The document recommends canonization or operational use without identifying evidence, dissent, risks, and review triggers.
- Reproducibility is claimed while required code, data, prompts, settings, or versions are unavailable.
- The review attacks contributors rather than claims, methods, mechanisms, incentives, or omissions.

Identifiers and Repository Practice

Stable identifiers make cross-model review and GitHub publication manageable. The repository may adopt a different naming convention, but IDs should be unique, parseable, versioned, and linked to source commits or release tags.

Artifact	Pattern	Example
Cross-review document	OAL-ACR-I-{iteration}-G{gate}-{AI}-v{X.Y}	OAL-ACR-I1-G1-CHATGPT-v0.1
Source artifact	OAL-{project}-{topic}-S{source}-{AI}-v{X.Y}	OAL-AIU-PROMPT-S3-CLAUDE-v0.2
Discrepancy	D-{two digits}	D-07
Assumption	A-{two digits}	A-12
Test/exercise	X-{two digits}	X-04
Improvement action	I-{two digits}	I-09
Architect decision	AD-{two digits}	AD-03
Hypothesis	{LAB}-H-{number or letter}	ECL-H-02
Red-team issue	RT-{two digits}	RT-05
Preregistration	PRE-{topic}-{date}-{version}	PRE-ENDCYCLE-2026-10-01-v0.1

Recommended Repository Bundle

- README or index describing the review scope and artifact relationships.
- DOCX and PDF rendered review documents.
- Machine-readable CSV or JSON registers when scale warrants them.
- Preregistration files with immutable commit references.
- Code, configuration, prompts, and environment notes for executed tests.
- Raw and processed outputs kept distinct.
- CHANGELOG recording revisions and status transitions.
- DECISIONS file or folder for Architect decisions and ratification records.
- DISSENT file or section preserving unresolved positions.
- LICENSE and contribution guidance appropriate to the repository.

Feedback and Cautions

The End Cycle Laboratory exercise demonstrates the value of running even small pilots against mature conceptual frameworks. Its transferable contribution is not that one review has the final answer, but that it exposes the distance between designing an exercise and actually executing one. The reusable process should preserve that distinction while remaining lighter than the work it evaluates whenever possible.

Constructive Criticisms

- A comprehensive cross-review can become self-protecting documentation. Tiering and stopping rules are necessary to prevent review from substituting for learning.
- A shared label set is useful only if examples, repository guidance, and periodic consistency checks keep collaborators from applying labels differently.
- Adversarial review should challenge claims, assumptions, methods, incentives, edge cases, and missing tasks—not contributors' motives or intelligence.
- Simulation findings and design recommendations must remain visibly separate, especially in executive summaries and public repository descriptions.
- Generator-evaluator separation is a strong default, but solo workflows need disclosed fallback safeguards rather than impossible independence requirements.
- A review should have a stopping rule tied to changed knowledge or action; another document is not automatically evidence of progress.
- The publication record should state what changed because of the review so future readers can see its operational value.
- Common prompts improve comparability but can induce convergence; distinct lanes and independent first passes protect analytical diversity.
- The protocol itself must be testable. Track completion time, number of actionable defects found, duplicate findings, unresolved decisions, and whether changes improve later outcomes.
- The protocol should permit retirement. Preserving every mechanism indefinitely can create the same accumulation problem the review process is intended to solve.

Questions for Team Review

- Are the five review tiers sufficient, or should project risk and canon status be separated into different dimensions?
- Which epistemic labels are essential, redundant, or missing?
- Who may assign, change, or close discrepancy resolution paths?
- What minimum evidence should be required before a framework advances from proposal to operational use?
- Which tests require human evaluation rather than additional AI review?
- How should the team handle inaccessible, proprietary, private, or safety-sensitive source artifacts?
- What repository metadata is mandatory for publication?
- When should dissent be preserved indefinitely, and when may it be archived as resolved?
- How should the team evaluate whether the protocol itself improves quality rather than merely increasing output?
- What authority, if any, should a cross-review body have beyond recommendation and documentation?

Reusable Document Template

The following structure is the required common core for a formal Tier 2–4 cross-review. Tier 0–1 reviews may use a reduced form while preserving the same claim discipline and status notices.

Section	Required content
Title Page	Document ID, title, version, date, preparer, prepared-for line, status, repository, source corpus window, review tier, and declared inputs.
Executive Summary	Topic reviewed; material actually reviewed; central adversarial conclusion; executed work; three to five key weaknesses or gaps; next action.
Table of Contents	Generated from document heading styles and updated before publication.
Mission and Directive	Review mission, operating directive, objectives, and non-goals.
Scope, Limits, and Non-Goals	Inside/outside scope, category of work, evidence use, authority, feasibility, and safety/privacy/legal/canon boundaries.
Source Corpus and Input Declaration	Complete artifact inventory, missing inputs, uncertain authorship, unavailable code/data, and secondhand descriptions.
Epistemic Labels	SOURCE, FINDING, EVIDENCE, ASSESSMENT, PROPOSAL, OPEN, LIMITATION, DECISION, HYGIENE, and NOT RUN.
Review Method	Questions, fairness standard, independence disclosure, and review limitations.
Topic or Framework Overview	Neutral summary of problem, users, mechanisms, outcomes, assumptions, dependencies, and implementation status.
Independent Source Analysis	Unique contributions, strengths, weaknesses, inconsistencies, omissions, unperformed tasks, and disposition recommendations.
Assumption and Dependency Audit	Structured register across technical, governance, ethical, operational, economic, narrative, and repository dimensions.
Discrepancy Register	Conflicts preserved with consequences, status, resolution path, and owner.
Testability Assessment	Claims classified as testable now, partly testable, not testable now, or not empirically testable.
Preregistration and Exercise Plan	Hypotheses, falsifiers, baselines, inputs, parameters, metrics, roles, thresholds, and limitations.
Results and Findings	PASS, FAIL, MIXED, INDETERMINATE, or NOT RUN with narrow interpretation and linked artifacts.
Iterative Improvement Register	Concrete actions, owners, priorities, closure evidence, target iteration, and status.
Architect Decision Points	Only nonempirical authority, canon, priority, scope, or value decisions.
What Changed	Specific modifications, retirements, quarantines, decisions, and open questions resulting from the review.
Next-Iteration Commission Prompt	Copy-ready prompt with source inputs, independence rule, stage gates, deliverables, quality gates, and non-canon status.
Appendices	Crosswalks, methods, raw-data references, prompts, source notes, version history, dissent, and reproducibility details.

Copy-Ready Commission Prompt

The prompt below may be adapted for each AI system. Bracketed fields should be completed before commissioning. Lane-specific instructions should be added without removing the common quality requirements.

STATUS: PROPOSED. NOT CANON. NOT ONESTAMP-RATIFIED. Document ID: OAL-ACR-I{iteration}-G1-{AI}-v0.1 Date: [Insert date] Prepared for: Nathan Lewis II, Architect of Record, Omniversal Architect Labs You are an independent collaborator conducting an adversarial cross-review for Omniversal Architect Labs. Review the supplied corpus concerning [TOPIC]. Your objective is not to create consensus, praise the corpus, or defend any prior document. Identify what the corpus establishes, what it only proposes, what it assumes, what it fails to test, where it contradicts itself, and what should be tested or decided next. Create a detailed document suitable for DOCX and PDF export. Include: 1. Title page with document ID, version, date, preparer, status, review tier, source window, and declared input list. 2. Executive summary and table of contents. 3. Mission statement or directive. 4. Scope, limits, and non-goals. 5. Source corpus inventory, including missing inputs, uncertain authorship, unavailable code/data, and secondhand descriptions. 6. Epistemic labels: SOURCE, FINDING, EVIDENCE, ASSESSMENT, PROPOSAL, OPEN, LIMITATION, DECISION, HYGIENE, and NOT RUN. 7. Neutral overview of the topic. 8. Independent review of each artifact: unique contribution, strengths, weaknesses, assumptions, inconsistencies, unperformed tasks, and recommendations to preserve, revise, quarantine, defer, or retire. 9. Assumption and dependency audit. 10. Discrepancy register. Do not average disagreements away. Assign TEST, DECISION, HYGIENE, DEFER, QUARANTINE, RETIRE, or PRESERVE. 11. Testability assessment identifying claims testable now, partly testable, not testable now, or not empirically testable. 12. At least one minimal validation exercise appropriate to the topic. If executed, provide preregistration, method, assumptions, controls, raw result summary, limitations, and reproducible artifacts. If no legitimate test can be run, state why. 13. Results table labeling every planned exercise PASS, FAIL, MIXED, INDETERMINATE, or NOT RUN. 14. Iterative improvement register with concrete actions, owners, closure evidence, and target iterations. 15. Architect decision points limited to questions not resolvable through testing, source verification, or documentation hygiene. 16. "What changed because of this review?" section. 17. Copy-ready prompt for the next iteration. 18. Appendices containing crosswalks, methods, artifact references, source notes, version history, and dissent where needed. Mandatory rules: - Do not represent a projection, prediction, designed behavior, or scenario expectation as a FINDING unless an exercise was actually executed. - Do not treat agreement among AI systems as evidence. - Do not use page count, framework complexity, institutional form, or number of definitions as proof of quality. - Do not silently interpret ambiguous source material as settled fact. Label new readings ASSESSMENT or PROPOSAL. - List every input used. Do not claim independence from materials you read. - Include a baseline, control, counterfactual, or comparison for every feasible exercise. - For classification, detection, ranking, or distinguishability, do not let the same unblinded collaborator create both ground truth and final evaluation without disclosure and independent checking. - Preserve dissenting positions and unresolved questions. - Separate real-world EVIDENCE, source/canon material, simulation FINDINGS, reviewer ASSESSMENTS, and Architect DECISIONS. - State limitations plainly and near the claims they constrain. - End with specific next actions, not generic recommendations. Required tone: rigorous, fair, adversarial but constructive, specific, version-controlled, and suitable for GitHub publication.

Appendix A — Blank Registers

Source Register

ID	Title	Preparer	Date/version	Type	Claimed stage	Actual stage	Location	Read?	Notes
S-01									
S-02									
S-03									

Discrepancy Register

ID	Discrepancy	Positions	Consequence	Status	Path	Owner
D-01						
D-02						
D-03						

Testability Register

Claim	Status	Method	Threshold	Control	Limitation	Result

Improvement Register

ID	Action	Trigger	Priority	Owner	Closure evidence	Iteration	Status
I-01							
I-02							
I-03							

Decision Log

ID	Question	Options	Consequences	Recommendation	Default	Trigger
AD-01						
AD-02						

Appendix B — Publication Checklist

- ☐ Document ID, version, date, preparer, status, and review tier are present.
- ☐ Status is marked PROPOSED, NOT CANON, and NOT ONESTAMP-ratified unless formally changed.
- ☐ Source window and every input actually used are declared.
- ☐ Missing and secondhand materials are identified.
- ☐ Executive summary distinguishes executed work from proposals and open questions.
- ☐ Table of contents is updated.
- ☐ Epistemic labels are applied consistently.
- ☐ Each source receives a fair summary before criticism.
- ☐ Assumptions, dependencies, omissions, and unperformed tasks are documented.
- ☐ Material disagreements appear in the discrepancy register.
- ☐ Resolution paths are assigned without forced averaging.
- ☐ Tier 2+ work includes an exercise or a documented reason none is legitimate.
- ☐ Preregistration predates confirmatory execution.
- ☐ Controls, baselines, counterfactuals, or comparisons are present where feasible.
- ☐ Generator-evaluator circularity is prevented or disclosed.
- ☐ Raw artifacts, code, prompts, settings, seeds, and model versions are preserved as applicable.
- ☐ Results use PASS, FAIL, MIXED, INDETERMINATE, or NOT RUN.
- ☐ Limitations appear next to the claims they constrain.
- ☐ Actions have owners, closure evidence, and target iterations.
- ☐ Architect decisions are limited to nonempirical authority, canon, priority, scope, or value questions.
- ☐ The document states what changed because of the review.
- ☐ Dissent and withdrawn positions are preserved.
- ☐ GitHub links, commit references, and artifact names are valid.
- ☐ DOCX and PDF outputs match materially.
- ☐ No placeholder text remains unless the artifact is intentionally published as a template.

Appendix C — Proposed Disposition

This v0.1 document is submitted for comparative pass-through. It should be reviewed alongside independent versions prepared by other AI systems and human collaborators. The team should record conflicts rather than merging them prematurely, identify elements suitable for a shared common core, and assign at least one test of the protocol itself before treating it as a standard OAL process.

Disposition item	Proposed status
Mission and directive	PRESERVE provisionally; invite alternate mission statements.
Tier model	TEST through at least two differently sized review subjects.
Epistemic labels	PRESERVE provisionally; run a reviewer consistency exercise.
Differentiated AI lanes	TEST against an identical-prompt condition for convergence and defect diversity.
Stage gates	PRESERVE for Tier 2–4; simplify for Tier 0–1.
Identifiers	HYGIENE review against existing OAL naming conventions.
Quality gates	Cross-review for burden, enforceability, and unintended exclusion.
Canon status	Remain PROPOSED until explicit Architect and ONESTAMP action, if applicable.